SymQ: Exact Equivalence Checking of Reversible Boolean Circuits with a Native Monomial-ZDD Engine — an Evaluation with Matched Drivers

Joshua Melka
Independent researcher (no affiliation)

Draft of 27 October 2026 — not submitted

Abstract

We study exact functional equivalence checking of classical reversible circuits built from X, CNOT, Toffoli and multi-controlled X gates, by symbolic simulation in the Boolean ring $\mathrm{GF}(2)[x_1, \ldots, x_n]/(x_i^2 - x_i)$: every output wire is kept in algebraic normal form (ANF), two circuits are equivalent iff their output polynomials coincide, and a minimum-degree monomial of a differing output is a counterexample that we re-check by concrete execution. The representation, zero-suppressed decision diagrams (ZDDs) over monomial families, is not new: PolyBoRi uses it for Boolean Gröbner bases. Our contribution is engineering and measurement: symq, an open-source checker with a small native C engine (hash-consed monomial ZDD, three-product multiplication, lossless operation cache, explicit budgets), and a protocol that runs every engine under the same driver and controls its caches. Under identical Python drivers and cold caches, on the 19 supported reversible circuits of the MQT QCEC test suite, the native engine computes the symbolic outputs 3.94× faster than PolyBoRi (geometric mean, 95% CI [3.49, 4.41], 19/19 circuits; 2.11× with warm caches) and 3.48× faster than CUDD BDDs; the per-call costs of the Python bindings, calibrated separately, do not explain these gaps. On a blind corpus of 30 generated circuits frozen before any measurement, the gap to PolyBoRi persists (3.47× [2.23, 5.04] on the 15 circuits both complete), the gap to CUDD does not (0.69× [0.15, 2.41]): CUDD, with dynamic reordering, completes more circuits and is faster on adders and multipliers. Sorting the factors of each product by size slows the engine down (0.90×); the gate loop fused in C is 2.05× faster than a Python loop over the same operations, a gain that our first harness hid. A reader who needs only a Boolean verdict should use a BDD or SAT checker; what this engine adds is the explicit ANF of every wire, its cost, and concretely re-checked counterexamples. All symbolic engines produce identical output polynomials, also under seven variable orders; the only disagreements, on the blind corpus, came from our conversion of CUDD BDDs to ANF, which we fixed. Limits: at most 64 variables; ANF blows up on multipliers under every order we tried; Python drivers; one machine; mostly sub-millisecond medians. We claim no new algorithm and no general superiority.

1 Introduction

Reversible circuits over X, CNOT (CX), Toffoli (CCX) and multi-controlled X (MCX) gates are the classical core of many quantum algorithms (arithmetic, oracles, uncomputation) [41, 17]. Their equivalence is a purely Boolean question: two circuits on n wires are equivalent iff they compute the same permutation of $\{0,1\}^n$. Tools from quantum design automation decide it among many other questions [10], and classical techniques (BDDs [9], SAT, algebraic methods) apply directly [40].

A particularly simple exact method is symbolic simulation in the Boolean ring: each wire carries a polynomial over $\mathrm{GF}(2)$ in algebraic normal form (ANF), a gate with controls C and target t updates $f_t \leftarrow f_t + \prod_{c \in C} f_c$, and canonicity of the ANF turns equivalence into equality of polynomials (Section 3). The cost is the size of the intermediate polynomials. Storing them as zero-suppressed decision diagrams [29] of their monomial families shares common sub-families, and is exactly what PolyBoRi does for Boolean Gröbner-basis computations [7].

This paper does *not* propose a new representation or a new decision procedure. It asks an engineering question and tries to answer it without fooling ourselves: *how much does a small, specialised native engine gain over mature native libraries on this task, once the driver and the caches are controlled?* Our contributions are:

1. **An exact, budgeted native engine** (Section 4) for monomial ZDDs specialised to reversible-circuit simulation, with concretely re-validated counterexamples, and its integration in an open-source checker with interchangeable exact backends.

2. **A matched-driver, cache-controlled protocol** (Section 5) that runs every engine under the same minimal Python driver and separates setup, symbolic computation, comparison, witness extraction and

explicit-monomial extraction, in cold and warm cache regimes, with paired statistics and a pre-registered blind corpus.

3. **Measurements, including adverse ones** (Section 6): the gains that survive matching and a blind corpus, the ones that do not (factor sorting; the advantage over CUDD), the families where the monomial representation fails, and a harness artifact that had hidden the benefit of our fused gate loop.

The phase-sensitive extension of `symq` to measurement-based uncomputation is a separate, unfinished line of work and is not evaluated here (Section 7).

2 Background and related work

Boolean polynomials and ZDDs. ZDDs [29] are reduced, ordered decision diagrams whose nodes with a 0 high child are suppressed; they represent families of sparse sets compactly and canonically [31]. Minato already used them to manipulate polynomials implicitly [30]. Representing a Boolean polynomial by the ZDD of its monomial family, with ring operations implemented as recursive ZDD operations, is the core of PolyBoRi [7], maintained today as BRiAl [36] and reachable from Python through a modular SageMath distribution [37]; its authors applied it to formal verification of circuits [8]. Our engine uses the same representation and the same kind of recursion; it differs in scope (at most 64 variables, one operation pattern), in engineering choices (Section 4), and in being budgeted. BDDs over Shannon cofactors [9], as in CUDD [35] (used here through `dd` [13]), represent the same Boolean functions without exposing their ANF; CUDD's ZDD products operate on covers, not on GF(2) sums, so CUDD offers no ANF ring.

Functional decision diagrams. The data structure itself is not new. Functional decision diagrams [24] decompose a function by the positive Davio expansion $f = f_0 \oplus x\,(f_0 \oplus f_1)$; with the rule that deletes a node whose Davio child is 0 [16], the reduced ordered FDD of f is, node for node, the ZDD of the monomial family of its ANF under the same order, as Minato observes [31] (`paper/tools/ofdd_vs_zdd.py` checks this exhaustively for every function of at most four variables). Kronecker FDDs mix Shannon and Davio decompositions per variable [15], and the sizes of BDDs and FDDs can differ exponentially in both directions [5]. What this paper studies is therefore not the diagram but its use: the GF(2) product (monomial multiplication with cancellation modulo 2, not the unate cube-set product of ZDD packages) driven gate by gate by reversible circuits, its cost, and how it compares with matched baselines. Positive-polarity Reed–Muller expansions are also the basis of reversible-logic synthesis [1, 18, 28], and ReVerC [4] compiles reversible circuits with a formally verified compiler.

Algebraic verification of arithmetic circuits. Computer-algebra verification of gate-level multipliers fights the monomial blow-up that we also observe, with backward rewriting and vanishing-monomial removal [27], combinations with SAT [22], or linear relations extracted from Gröbner bases [21]. Our forward simulation uses none of these techniques and is not competitive on multipliers; we do not compare with these tools, which target And-Inverter Graphs rather than reversible circuits.

Equivalence checking of reversible and quantum circuits. Equivalence of reversible circuits has been decided with decision diagrams and SAT [40]; RevLib [41] provides benchmarks. For quantum circuits, decision-diagram checkers go back to QuIDD [38]; MQT QCEC [10, 11, 33] combines decision-diagram, simulation and ZX checkers. On X/CX/MCX circuits, unitary equality coincides with functional equality, so QCEC is a natural point of comparison even though it answers a more general question and builds no polynomial. Path sums [2] carry, for reversible gates, exactly our ANF outputs together with a phase polynomial; phase-based methods optimise and check larger gate sets [3]. QuPRS combines path-sum reduction with weighted model counting [20], and FeynmanDD reduces path-integral questions to counting with classical decision diagrams [39]. Closest to our representation, Irene [23] checks hybrid quantum programs, represents Boolean expressions as XOR–AND graphs rather than ANF, and reports, counting ANF monomials with ZDDs, that explicit ANF would need more than 6 GiB on 30 of its program pairs: independent evidence that ANF is the wrong representation for some workloads. Measurement and feed-forward are handled by dedicated checkers [19, 12, 34] and by outcome-complete stabilizer simulation [25, 26]; they are outside our scope.

What is new here. `paper/CLAIMS.md` lists each claim with its prior art and the experiment that could refute it. In short: the representation and the product recursion are known; a specialised, budgeted native engine and, above all, an evaluation that controls driver and cache effects are what we contribute. We found no prior evaluation of a monomial-ZDD engine for reversible-circuit equivalence against PolyBoRi and CUDD under matched drivers, but our search (arXiv full-text queries and the references above) is not exhaustive.

3 Problem and semantics

Definition 1 (Circuits)**.** A circuit of width n is a list of gates (C, t) with $C \subseteq \{0, \ldots, n-1\}$ and $t \notin C$. On a state $s \in \{0,1\}^n$ the gate sets $s_t \leftarrow s_t \oplus \prod_{c \in C} s_c$ (X: $C = \emptyset$; CX: $|C| = 1$; CCX: $|C| = 2$; MCX otherwise).

Let $B_n = \mathrm{GF}(2)[x_0, \ldots, x_{n-1}]/(x_i^2 - x_i)$. Each element has a unique ANF, a sum of distinct square-free monomials, identified with a family of subsets of $\{0, \ldots, n-1\}$; addition is symmetric difference of families.

Fact 1 (canonicity)**.** *B_n is isomorphic to the ring of Boolean functions $\{0,1\}^n \to \{0,1\}$; two polynomials denote the same function iff their monomial families are equal.*

Symbolic simulation starts from $f_i = x_i$ and applies $f_t \leftarrow f_t + \prod_{c \in C} f_c$ for each gate.

Proposition 1 (soundness and completeness)**.** *For every input a, $(f_0(a), \ldots, f_{n-1}(a))$ is the circuit's output on a. Hence two circuits are equivalent iff their output families are equal wire by wire.*

Proof. Induction on gates: evaluation at a is a ring homomorphism $B_n \to \mathrm{GF}(2)$, so $(f_t + \prod f_c)(a) = s_t \oplus \prod s_c$; combine with the fact. □

Proposition 2 (minimum-degree witness)**.** *If $p \neq 0$ and M is a monomial of p of minimum degree, then $p(\chi_M) = 1$, where χ_M sets exactly the variables of M to 1.*

Proof. $p(\chi_M)$ is the parity of the number of monomials of p contained in M. A monomial strictly contained in M would have smaller degree, so only M itself counts. □

Applied to $f_i^L + f_i^R$ for the first differing output i, Proposition 2 yields an input on which the circuits differ. symq re-executes both circuits on it and raises an internal-inconsistency error if they agree, so a faulty backend can never return an unchecked counterexample. Equivalence verdicts have no such independent check; they rest on the propositions above and on differential testing (Section 5). Full statements, including budgets, are in `paper/FORMAL_MODEL.md`; none of them is machine-checked.

4 The native engine

Representation. Nodes are triples (ℓ, lo, hi) in a single array, hash-consed through an open-addressing unique table; node 0 is the empty family (0), node 1 is $\{\emptyset\}$ (1). A node at level ℓ with variable $v = x_{\pi(\ell)}$ denotes $lo + v \cdot hi$; zero suppression forbids $hi = 0$; both children lie at strictly larger levels (or are terminals), which is what makes the representation canonical. Children are always created before their parent, so they have smaller identifiers, and every bottom-up quantity (monomial count, minimum-degree monomial) is computed by a single pass in identifier order, without recursion.

Operations. Addition is the symmetric-difference recursion. Multiplication splits both operands on the top variable v, with $P = P_0 + vP_1$ and $Q = Q_0 + vQ_1$:

$$PQ = P_0Q_0 + v\,(P_0Q_1 + P_1Q_0 + P_1Q_1), \tag{1}$$

which follows from $v^2 = v$. Terminal cases include $P \cdot P = P$ (every element of B_n is idempotent) and $P + P = 0$. An algebraically equivalent two-product recurrence keeps the low term, $P_0Q_0 + v\big((P_0 + P_1)(Q_0 + Q_1) + P_0Q_0\big)$; its high part alone is not the product, and with $P_0 = Q_0 = 1$, $P_1 = Q_1 = 0$ it gives 0 instead of 1. The recurrence built up to two orders of magnitude more nodes on one of our circuits under a reversed variable order, so we use (1); we make no asymptotic claim about either form.

Engineering choices. (i) A lossless operation cache: entries are never evicted and the table grows, unlike the lossy computed tables of general-purpose packages; managers are short-lived (one per verification), so no garbage collection is performed. (ii) Monomial counts are stored per node at creation as signed 64-bit integers. A 64-variable family can have up to 2^{64} monomials, and in the measured version such a count can overflow, which is undefined behaviour in C (with the compiler used, in practice a wrap-around that no check detects); we found this defect while preparing this paper. Its fix (saturating counts, and an error when a saturated polynomial is materialised) is in the repository but *not* in the measured binary. Counts drive budgets, factor ordering, extraction and the choice of representation in symq's adaptive backend, never a verdict (equality is node identity), and the largest output we measured has 67 108 911 monomials. (iii) Monomials are 64-bit masks, which bounds the width to 64; in the measured version, importing explicit monomials at width exactly 64 rejected every non-constant monomial (an undefined 64-bit shift), a path that simulation does not use; fixed since. (iv) A `simulate` entry point takes the whole gate list (control indices, offsets, targets) and runs the loop

in C, multiplying the controls of each gate in increasing monomial count. (v) Budgets on time (checked every 1 024 recursive steps and every 64 gates), nodes (at creation) and monomials (when a polynomial is materialised) raise an exception; no partial result is ever returned. The engine is about 1 240 lines of C against the CPython API and is optional: without a compiler, the pure-Python backends remain.

5 Evaluation methodology

Questions. (Q1) How fast is the native engine relative to PolyBoRi and CUDD when only the engine *and its Python binding* differ (the driver being identical)? (Q2) Which parts of the engine and driver matter (ablation)? (Q3) Does the comparison survive circuits that influenced no design decision? (Q4) Are all results exact?

Drivers. Every engine is run under the same drivers: (a) *minimal loop*: the same Python loop for all tools, which multiplies the controls of each gate in gate order and adds the product to the target, with no guard and no counter; only the two ring operations differ (for PolyBoRi, Python's `*` and `+` on `BooleanPolynomial`; for CUDD through `dd` [13], conjunction and `xor`); (b) *minimal loop, sorted*: the same with factors sorted in Python by monomial count (`symq`: its stored count; PolyBoRi: `len`); (c) *`symq`'s `Algebra` driver*, which decodes each gate and applies its guards and counters inside the timed phase; (d) for `symq` only, the fused C loop that it ships, fed control, offset and target arrays built before the timer (as the gate list of the loops is). Equality is node identity for `symq` and CUDD and `(a+b).is_zero()` for PolyBoRi (Sage's `==` compares term by term and is an order of magnitude slower on large outputs). The loop calls the engines through their Python bindings (a C extension for `symq`, Cython for PolyBoRi and `dd`), whose per-call costs differ; we measure them separately (Section 6) rather than attribute the whole loop time to the engine.

Cache regimes. SageMath caches `BooleanPolynomialRing` objects by their arguments: two rings of the same width are the same object, with the same operation cache. In our earlier harness every PolyBoRi repetition after the first therefore reused one ring and its caches (an inference from Sage's caching behaviour, which we did not measure in those runs), while every other engine started from a fresh manager. We now measure two regimes for every engine: *cold* (a fresh manager per run; for PolyBoRi, a ring with fresh variable names) and *warm* (one manager reused after a filling run, i.e. re-verification). Setup (manager and variables) is timed separately from the symbolic computation.

Corpora. (1) The 19 supported circuits of MQT QCEC's `test/circuits/original` (16–35 wires; one circuit with controlled-V gates is rejected), imported byte for byte with hashes and license [33]. They have influenced our design, so they are not a holdout. (2) A *blind* corpus of 30 circuits generated by a seeded script and frozen by a hash manifest before any symbolic measurement: Cuccaro adders [14] and controlled adders up to 31 bits, shift-and-add multipliers up to 10 bits, incrementers up to 64 bits, and seeded random MCX circuits of 16, 32 and 64 wires. Hypotheses and analysis were pre-registered; one amendment (cache regimes, timeout censoring, parallel workers) was recorded before any blind measurement was inspected. Workloads: *simulate* (symbolic outputs), *equiv* (against a variant built from sound rewrites) and *nonequiv* (against the circuit with one inserted MCX gate).

Measurement. One fresh worker process per (circuit, workload, mode), pinned to one core of a 4-core Intel Xeon VM with about 15 GiB of memory shared by all processes (CPython 3.12.3; extension built with GCC 13.3.0 and `-O3`[1]). Every run uses two workers on two dedicated cores (isolated re-runs: one); the blind run shared the machine with an earlier QCEC-corpus run (superseded) and lighter jobs, the other runs with lighter jobs only. One warmup run, then the median of 7 runs (3 above 1 s, 1 above 10 s); 120 s per cell; cells of one family (random circuits: of one gate and control count) run in increasing size and larger ones are censored after a timeout. Ratios are paired per circuit; we report geometric means with 95% percentile-bootstrap intervals over circuits, the range and the number of wins. Explicit monomial extraction is timed once and feeds an output fingerprint compared across all modes. Versions: PolyBoRi through passagemath-brial 10.8.13, CUDD 3.0.0 through `dd` 0.6.0, MQT QCEC 3.11.0 restricted to its complete alternating checker (no simulation or ZX checker, so every conclusive verdict is a proof).

Exactness checks. Beyond the cross-mode fingerprints, the test suite of the measured commit (2 490 tests) compares every backend, the native one included, with concrete execution on all inputs of seeded random circuits of up to 7 wires, checks the native engine against the Python ZDD node for node, and shows that the same differential checks catch deliberately broken Python algebras (missing cross term, no cancellation, OR instead

[1] The records of the blind run list the output of `cc -version` (clang 18), not the compiler that built the extension; the measured binary, identified by its SHA-256, carries GCC 13.3.0 in its `.comment` section.

Table 1: Matched drivers, cold caches, compute phase. Geometric mean of the paired ratio baseline/candidate (> 1: symq faster) with a 95% bootstrap interval; in parentheses, wins/circuits completed by both, and circuits not completed (n/c) by at least one side. Modes: nat-loop: symq native, minimal loop; pb-loop: PolyBoRi, minimal loop; nat-sorted: symq native, minimal loop, sorted; pb-sorted: PolyBoRi, minimal loop, sorted; nat-fused: symq native, fused C loop; pb-best: PolyBoRi, best of its three drivers; bdd-loop: CUDD BDD, minimal loop (reordering); bdd-static: CUDD BDD, minimal loop (static order).

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-loop / pb-loop	3.94 [3.49, 4.41] (19/19)	3.00 [2.61, 3.42] (19/19)	3.46 [3.12, 3.80] (19/19)
nat-sorted / pb-sorted	4.09 [3.66, 4.50] (19/19)	3.12 [2.75, 3.49] (19/19)	3.55 [3.15, 3.98] (19/19)
nat-fused / pb-best	8.07 [6.67, 9.68] (19/19)	6.15 [4.44, 8.39] (19/19)	8.06 [6.55, 9.82] (19/19)
nat-loop / bdd-loop	3.48 [2.95, 4.10] (19/19)	2.74 [2.27, 3.26] (18/19)	3.15 [2.72, 3.64] (19/19)
nat-loop / bdd-static	3.38 [2.87, 3.96] (19/19)	2.71 [2.22, 3.26] (19/19)	3.10 [2.69, 3.58] (19/19)
blind (30 circuits)			
nat-loop / pb-loop	3.47 [2.23, 5.04] (13/15; 15 n/c)	3.34 [2.15, 4.96] (12/14; 16 n/c)	3.00 [1.94, 4.22] (14/15; 15 n/c)
nat-sorted / pb-sorted	3.98 [2.46, 5.89] (13/15; 15 n/c)	3.30 [2.12, 4.70] (12/14; 16 n/c)	3.73 [2.28, 5.50] (14/15; 15 n/c)
nat-fused / pb-best	5.03 [2.78, 8.52] (13/15; 15 n/c)	5.04 [2.70, 8.66] (12/14; 16 n/c)	4.32 [2.46, 7.07] (14/15; 15 n/c)
nat-loop / bdd-loop	0.69 [0.15, 2.41] (9/16; 14 n/c)	1.08 [0.34, 2.91] (9/14; 16 n/c)	0.64 [0.13, 2.31] (9/16; 14 n/c)
nat-loop / bdd-static	1.63 [0.45, 4.52] (11/15; 15 n/c)	1.38 [0.41, 3.60] (10/14; 16 n/c)	1.32 [0.29, 4.07] (11/15; 15 n/c)

Table 2: Ablation of the native engine (cold compute phase). Each row changes one thing: the engine (C instead of Python, same driver), the driver (minimal loop instead of symq's `Algebra` layer), factor sorting, and fusing the loop in C. Ratios below 1 are slowdowns. Modes: nat-alg: symq native, `Algebra` driver; py-alg: symq Python ZDD, `Algebra` driver; nat-loop: symq native, minimal loop; nat-sorted: symq native, minimal loop, sorted; nat-fused: symq native, fused C loop.

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-alg / py-alg	5.41 [4.48, 6.73] (19/19)	4.19 [3.53, 5.09] (19/19)	4.29 [3.61, 5.26] (19/19)
nat-loop / nat-alg	2.13 [1.94, 2.33] (19/19)	2.11 [1.85, 2.39] (19/19)	2.40 [2.18, 2.61] (19/19)
nat-sorted / nat-loop	0.90 [0.87, 0.93] (2/19)	0.89 [0.87, 0.92] (0/19)	0.88 [0.86, 0.90] (0/19)
nat-fused / nat-sorted	2.28 [2.02, 2.54] (19/19)	2.30 [1.85, 2.83] (19/19)	2.67 [2.30, 3.07] (19/19)
nat-fused / py-alg	23.58 [21.31, 25.84] (19/19)	18.19 [15.30, 21.33] (19/19)	24.06 [21.26, 26.71] (19/19)
blind (30 circuits)			
nat-alg / py-alg	7.34 [5.95, 9.08] (15/15; 15 n/c)	6.21 [4.75, 8.38] (13/13; 17 n/c)	7.39 [6.04, 8.93] (14/14; 16 n/c)
nat-loop / nat-alg	1.45 [1.20, 1.75] (12/16; 14 n/c)	1.65 [1.37, 1.98] (14/14; 16 n/c)	1.45 [1.22, 1.72] (13/16; 14 n/c)
nat-sorted / nat-loop	0.92 [0.89, 0.96] (2/16; 14 n/c)	0.92 [0.88, 0.96] (2/14; 16 n/c)	0.95 [0.89, 1.01] (7/16; 14 n/c)
nat-fused / nat-sorted	1.55 [1.26, 1.92] (14/16; 14 n/c)	1.74 [1.35, 2.24] (12/14; 16 n/c)	1.50 [1.22, 1.87] (14/16; 14 n/c)
nat-fused / py-alg	16.11 [11.83, 21.77] (15/15; 15 n/c)	17.59 [13.58, 22.11] (13/13; 17 n/c)	16.81 [12.60, 22.22] (14/14; 16 n/c)

of XOR). Runs of the native tests under AddressSanitizer and UndefinedBehaviorSanitizer were added together with the overflow fix and are not part of the measured commit. These are tests, not proofs: no part of the engine is machine-checked.

6 Results

Exactness (Q4). A group is one (circuit, workload) cell across all symbolic modes. For *simulate* the compared fingerprint is a hash of the explicit output monomials (the polynomials themselves); for *equiv* and *nonequiv* it is only the verdict and the index of the first differing output, and every counterexample is re-checked concretely. On the QCEC corpus, 57 groups were compared with 0 disagreement; QCEC returned 38 correct proofs. On the blind corpus, 72 output groups were compared: 3 fingerprint disagreements, 0 wrong verdicts, 0 invalid witnesses. No counterexample failed its concrete re-check. The 3 disagreements (*simulate* on `mult_6`, `rand_16_100_2`, `rand_16_100_4`) all set CUDD with dynamic reordering against every other engine, CUDD with a static order included. The BDDs were right (on `rand_16_100_2` they agree with concrete execution on 300 random inputs); our Reed–Muller conversion of BDDs to ANF, used only for fingerprints, memoised transforms by node identifier without keeping the nodes alive, so CUDD reused freed nodes for other functions. A regression test now reproduces this, and the fix keeps the nodes referenced; timings are unaffected, since conversion is outside the timed phase. All engines that build ANF agree on every group.

Matched comparison (Q1). Table 1 is the main result. With the same minimal loop and cold caches, the native engine is 3.94× faster than PolyBoRi on *simulate* over the QCEC corpus and wins on all 19 circuits; equivalence and non-equivalence checks, which also simulate a second circuit and compare, show 3.00× and 3.46×. With warm caches (one manager reused after a filling run) the ratio drops to 2.11× [2.03, 2.19]: part of the cold-cache gap is the cost of filling PolyBoRi's caches. Against CUDD BDDs, which represent functions rather than monomials, the native engine is 3.48× faster with dynamic reordering and 3.38× with a static

Table 3: Variable-order sensitivity of the native engine (*simulate*, fused loop, cold): identity, reversed and five seeded random orders per circuit. Node spread: largest over smallest number of ZDD nodes across the orders of one circuit. Fingerprints: hashes of the explicit output monomials, which must not depend on the order.

corpus	complete orders	equal fingerprints	node spread geo. / max	identity/best time (geo.)	identity best
QCEC (19 circuits)	19/19	19/19	1.79 / 146 (`c2_181`)	1.19	6/19
blind (30 circuits)	17/30	17/17	4.52 / 85 (`adder_16`)	3.13	0/17

order. Fixed costs are timed separately: a fresh PolyBoRi ring costs 4.387 ms and a CUDD manager 3.008 ms (medians over the QCEC corpus), against 0.009 ms for the native manager; they dominate end-to-end times on these small circuits, which is why we do not headline end-to-end ratios.

Binding costs. The minimal loop calls each engine through its Python binding. We calibrated one product and one sum of two variables, a cache hit for every engine, in a tight loop (run `33515a36080b`): 56/50 ns for the native engine, 156/117 ns for PolyBoRi and 70/224 ns for CUDD (product/sum), against 23 ns for an empty Python call. Subtracting these costs, times the number of calls of each circuit, from both sides is a *sensitivity analysis*, not a bound: the calibrated call is a cache hit, so the subtraction also removes a cache lookup, and it assumes that a call costs the same whatever its operands. It leaves ratios of 4.41× against PolyBoRi and 3.86× against CUDD on the QCEC corpus (19/19 and 19/19 pairs retained; a pair is dropped when a corrected time is not positive). It is evidence that the bindings do not explain the gap, not a proof: the bindings take a median 27% of the native loop time and 16% of PolyBoRi's, and a native driver for PolyBoRi or CUDD remains the clean answer (Section 7).

Ablation (Q2). Table 2 decomposes the distance between the Python ZDD and the shipped engine. The C core alone, under `symq`'s `Algebra` driver, gives 5.41× cold (but only 1.21× warm, when nearly every operation is a cache hit and the driver dominates). Replacing that driver by the minimal loop gives another 2.13×: `symq`'s per-gate guards and counters are a large share of the cost of small circuits. Sorting factors by monomial count, which our fused loop does, *slows* the engine (0.90×); fusing the loop in C then gains 2.28×. The shipped fused loop against the unsorted minimal Python loop is 2.05× (95% CI [1.83, 2.26], 19/19 wins). Our first matched harness found no gain (0.97×): it built the fused loop's index arrays and gathered statistics inside the timed phase, while the Python loops received a pre-built gate list. Fed the same pre-built input, fusion removes about half of the time of these small circuits, consistent with the binding costs above; sorting is the part that does not pay.

Variable order (robustness). Table 3 varies the variable order of the native engine (runs `33515a36080b`; identity, reversed and five seeded random orders; 25 s and $2 \cdot 10^7$ nodes per cell; family-wise censoring on the blind corpus). The output monomials are identical under every order that completed, as they must be (0 and 0 disagreements). On the QCEC corpus the default (identity) order is within 1.19× of the best of the seven in time (geometric mean) and the best on 6/19 circuits, although node counts vary by up to 146× (`c2_181`). The blind corpus is different: the identity order is never the best, is 3.13× slower than the best order on the 17 circuits where all orders complete, up to 155.8× on `adder_16`, and exhausts the budget on `adder_24`, `adder_31`, `cadder_24`, `cadder_30` where some random order completes. The adders that fail under the default order are thus an *order* effect, not a limit of the representation; the multipliers beyond 5 bits and the dense random circuits fail under all seven orders or are censored after a smaller circuit of their family failed (9 circuits). In the matched comparison every monomial engine uses the index order (PolyBoRi's ring too) and CUDD's dynamic reordering is the only adaptive ordering measured.

Blind corpus (Q3). Table 4 gives the completion of every mode, Table 5 the paired ratios by family and Table 1 the aggregates. Ratios are computed only on circuits both tools complete, a selection that can favour either side; coverage is reported next to them. *H1* (native faster than PolyBoRi under the same loop, on *simulate*, over the circuits both complete): 3.47×, 95% CI [2.23, 5.04], 13/15 wins; supported. *H2* (fused loop faster than the minimal loop; corrected harness): 1.52×, 95% CI [1.30, 1.80], 15/18 wins; supported. *H3* (exactness): rejected. *H4* (multipliers defeat monomial representations sooner than BDDs): of 7 multipliers, *simulate* completes on 4 with the native loop (largest `mult_5`), 4 with PolyBoRi (`mult_5`) and 5 with CUDD (`mult_6`). On *simulate*, the native loop / PolyBoRi / CUDD with reordering complete 4/3/5 of 5 adders, 3/3/5 of 5 controlled adders, 4/4/5 of 7 multipliers, 4/4/4 of 4 incrementers and 1/2/5 of 9 random circuits.

Every ratio below is conditional on the circuits *both* tools complete; the denominators are attached to each number, and completion is reported separately (Table 4, Figure 1), because a censored or killed cell is not a slow one. Against PolyBoRi the result of the QCEC corpus carries over: same loop, same order, 3.47× [2.23,

Table 4: Completion on the blind corpus (90 cells per mode: 30 circuits × 3 workloads; QCEC has no *simulate*). *censored*: not run because a smaller circuit of the same family timed out (120 s); *killed*: worker killed by the memory cgroup of the shared machine (cells of the original run: also when re-run alone; fused cells: not re-run); *error*: any other failure.

mode	ok	timeout	censored	killed	error
symq native, fused C loop	52	13	19	6	0
symq native, minimal loop	46	12	32	0	0
PolyBoRi, minimal loop	46	12	32	0	0
PolyBoRi, `Algebra` driver	46	12	32	0	0
CUDD BDD, minimal loop (reordering)	71	6	13	0	0
CUDD BDD, minimal loop (static order)	63	12	15	0	0
symq Python ZDD, `Algebra` driver	42	12	36	0	0
symq sparse ANF (Python)	45	7	32	6	0
MQT QCEC (alternating checker)	51	5	4	0	0

Table 5: Blind corpus by family, *simulate*, cold compute phase: geometric mean of the paired ratio (completed by both / circuits in the family). *extra nat*: circuits the native minimal loop completes and the baseline does not; *extra bdd*: the reverse against CUDD.

family	vs. pb-loop	vs. bdd-loop	extra nat	extra bdd
`adder`	5.10 (3/5)	0.25 (4/5)	1	1
`cadder`	4.00 (3/5)	1.23 (3/5)	0	2
`incr`	5.61 (4/4)	4.06 (4/4)	0	0
`mult`	2.24 (4/7)	0.23 (4/7)	0	1
`rand`	0.60 (1/9)	0.47 (1/9)	0	4

5.04] on 15 of 30 pairs, faster on every family where both complete something except the single random circuit. Against CUDD it does not. With dynamic reordering CUDD completes 71 of 90 cells, the native loop 46; the native engine stays faster on incrementers and controlled adders and is several times slower on adders and multipliers, over 16 of 30 pairs (8 circuits where CUDD completes and the native loop does not, 0 the other way), and the aggregate interval contains 1. Circuits within one family are generated size variants rather than independent draws, so we also report a family-balanced mean that gives each family one vote, with a bootstrap that resamples families: 2.74× [1.94, 5.11] over 5 families against PolyBoRi, and 0.67× [0.25, 2.21] against CUDD, the same conclusion with honest widths. The adders show the mechanism: they fail under the index order that every monomial engine uses, but complete under some random order (Table 3), and CUDD reorders dynamically; against CUDD with a static order the ratio is 1.63× [0.45, 4.52]. The advantage over CUDD measured on the QCEC corpus is therefore a property of small circuits under a good default order, not of the representation. H2 is supported once the harness is corrected, and H4 points in the expected direction on seven multipliers, too few for a test. H3, as pre-registered, is rejected, by the CUDD conversion defect described under Q4, not by an engine that builds ANF.

Deviations from the pre-registration. The corpus, the hypotheses and one amendment were public before the run started; the departures below all happened afterwards and are recorded cell by cell in `paper/REPRODUCIBILITY.md`, with the raw runs, the isolated re-runs and `merge.json` sealed in the repository. In short: the cell-status log was read during the run, and an exploratory variable-order experiment was run alongside it; H4 names no test; censoring grouped random circuits by name prefix; sixteen cells were killed by the memory cgroup of the shared machine (they were re-run alone: one completed, six timed out, two were censored, seven were killed again); a ten-second source-tree replacement during the run forced fourteen cells and two neighbours to be re-measured; and every fused cell was re-measured with the corrected harness. Killed cells are reported as killed, never as timeouts.

MQT QCEC. Table 6 reports QCEC for context, end to end on both sides (conversion and check for QCEC; manager setup, both simulations, comparison and witness for symq). On the QCEC corpus the ratio is 61.83× on *equiv* (95% CI [36.05, 105.87]). No QCEC verification takes less than 2.8 ms, while the median native one takes 0.05 ms: on these small circuits the ratio mostly measures QCEC's fixed cost and says nothing about larger circuits. The comparison is not matched: QCEC decides unitary equality for general quantum circuits and builds no polynomial, and its default portfolio, which we disabled to keep only complete proofs, may be faster.

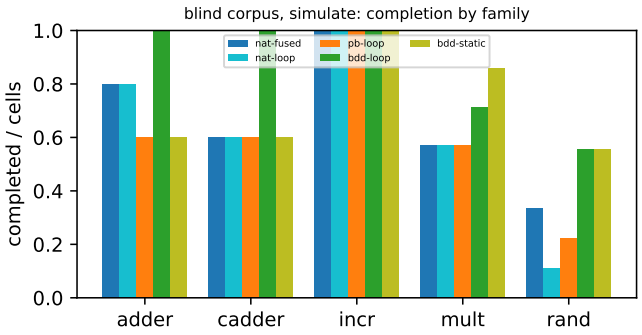


Figure 1: Completion is part of the result, not a footnote: fraction of *simulate* cells that finished (120 s limit, censoring and kills included in the denominator) on the blind corpus, by family and mode. The scatter (Figure 2) can only show circuits both tools completed; this figure shows what that selection hides.

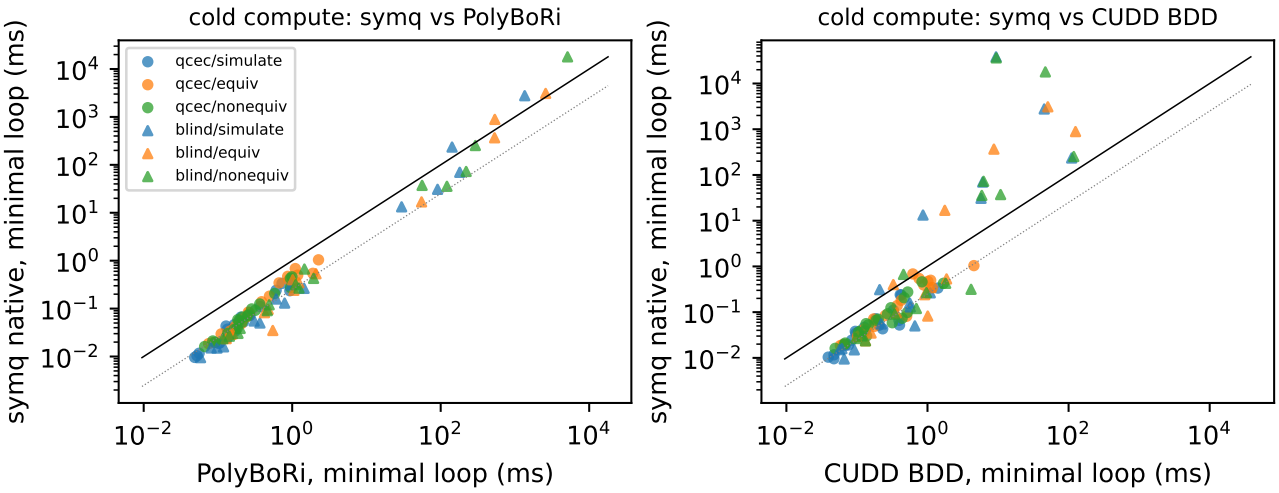


Figure 2: Per-circuit cold compute time of the native engine against PolyBoRi (left) and CUDD BDD (right), same minimal loop, all workloads, circuits completed by both. A constant factor over two orders of magnitude of running time is what a per-call cost would produce, which is why we calibrate the bindings separately. Solid: equal time; dotted: 4× faster.

7 Threats to validity and limitations

- **Single machine, small circuits.** Medians are often below one millisecond; one VM; no CPU frequency control. Paired ratios and bootstrap intervals over circuits do not remove machine effects.
- **Drivers and bindings.** The minimal loop is Python for every engine, and the bindings differ in per-call cost. We calibrate that cost and bound its effect (Section 6), but a native driver for PolyBoRi (its C++ API) or CUDD (its C API) on the same gate arrays is the only clean answer and is future work.
- **Memory and caches.** The native engine never evicts its operation cache and never collects garbage; CUDD runs with garbage collection and `dd`'s default cache limits; PolyBoRi with its defaults. We report no memory comparison (the recorded peak-RSS growth is not comparable across tools), and in the blind corpus a native cell can die from memory exhaustion rather than time out. Part of the cold/warm difference may be page faults and lazy initialisation inside the timed phase rather than cache contents.
- **Missing baselines.** We compare with no SAT-based combinational equivalence checker (e.g. ABC's `cec` [6], which uses SAT sweeping [32], on a miter) and no path-sum verifier (Feynman [2]); both are natural answers to the cases where ANF explodes.
- **Configuration.** PolyBoRi uses the `lp` order; CUDD its defaults, with and without dynamic reordering; no tuned variable order for anyone.
- **Representation and order.** Width at most 64; monomial representations explode on multipliers (Tables 4 and 3); the default variable order fails on adders that other orders handle, and we have no ordering heuristic; no garbage collection, so memory grows with the number of operations of a verification.

Table 6: End-to-end time of one verification: symq native (manager setup, both simulations, comparison, witness) against MQT QCEC's alternating decision-diagram checker (circuit conversion and check, default construction of its own managers). QCEC answers unitary equality and builds no polynomial; *simulate* does not apply. Modes: nat-fused: symq native, fused C loop; qcec: MQT QCEC (alternating checker).

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-fused / qcec	n/a	61.83 [36.05, 105.87] (19/19)	63.63 [42.65, 90.88] (19/19)
blind (30 circuits)			
nat-fused / qcec	n/a	11.74 [1.82, 58.31] (12/16; 14 n/c)	4.97 [0.55, 29.58] (14/17; 13 n/c)

- **Corpus.** The QCEC circuits influenced the design; the blind corpus is generated, not gathered from independent sources. Many QCEC test circuits come from RevLib; we rely on the licence of the QCEC repository and have not traced each file's provenance.
- **Scope.** Classical reversible gates only. symq also contains a specification and reference implementation of a restricted phase fragment (± 1 phases, X-basis measure-and-reset, single-outcome Z feed-forward) aimed at measurement-based uncomputation [17]; it has proof sketches and exhaustive small-case tests against an independent state-vector oracle, but no external corpus and no budgeted backend, and is not a contribution of this paper. Outcome-complete phase simulation and hybrid-circuit checkers [26, 34, 23] address related, broader questions.

8 Conclusion

A specialised native monomial-ZDD engine computes the ANF outputs of small reversible circuits several times faster than PolyBoRi when both are driven identically with controlled caches, on the QCEC test circuits and on a blind corpus; calibrated binding costs do not explain the gap. Against CUDD BDDs the advantage seen on the QCEC circuits does not survive the blind corpus: it depends on the family and on a variable order that the monomial engines do not adapt. Fusing the gate loop in C pays, sorting factors by size does not, and our own harness had hidden the first effect. These are engineering results on one machine and two small corpora, not a claim of algorithmic novelty or general superiority.

Artifact and reproducibility

Code, corpora, sealed results (with SHA-256 manifests) and the scripts that generate every table and figure of this paper are in the symq repository; `paper/REPRODUCIBILITY.md` gives the commands. Runs: QCEC corpus `33515a36080b`; blind corpus `23469829a0dc` (merged with its re-runs and the corrected fused runs, `benchmarks/merge_runs.py`); variable orders `33515a36080b`; bindings `33515a36080b`. All use the same native binary (SHA-256 prefix `2e3f6791`), built from sources that predate the two fixes of Section 4 and later changes to the engine (smaller nodes, an optional bounded cache, a gate loop that releases Python's global interpreter lock). The commit, tag and DOI will be recorded here once the author archives a release; nothing has been released at the time of writing.

Acknowledgements and disclosure

The code and the experiments were carried out with the help of several AI systems, which also assisted in drafting this manuscript. This work received no funding; the author declares no competing interests.

References

[1] Abhinav Agrawal and Niraj K. Jha. Synthesis of reversible logic. In *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, volume 2, pages 1384–1385. IEEE Computer Society, 2004.

[2] Matthew Amy. Towards large-scale functional verification of universal quantum circuits. In *Proceedings 15th International Conference on Quantum Physics and Logic (QPL 2018)*, volume 287 of *Electronic Proceedings in Theoretical Computer Science*, pages 1–21, 2019.

[3] Matthew Amy, Dmitri Maslov, and Michele Mosca. Polynomial-time T-depth optimization of Clifford+T circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(10):1476–1489, 2014.

[4] Matthew Amy, Martin Roetteler, and Krysta M. Svore. Verified compilation of space-efficient reversible circuits. In *Computer Aided Verification (CAV 2017), Part II*, volume 10427 of *Lecture Notes in Computer Science*, pages 3–21. Springer, 2017.

[5] Bernd Becker, Rolf Drechsler, and Ralph Werchner. On the relation between BDDs and FDDs. *Information and Computation*, 123(2):185–197, 1995.

[6] Robert Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification (CAV 2010)*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010.

[7] Michael Brickenstein and Alexander Dreyer. PolyBoRi: A framework for Gröbner-basis computations with Boolean polynomials. *Journal of Symbolic Computation*, 44(9):1326–1345, 2009.

[8] Michael Brickenstein, Alexander Dreyer, Gert-Martin Greuel, Markus Wedler, and Oliver Wienand. New developments in the theory of Groebner bases and applications to formal verification. arXiv:0801.1177 [math.AC], 2008.

[9] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.

[10] Lukas Burgholzer and Robert Wille. Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(9):1810–1824, 2021.

[11] Lukas Burgholzer and Robert Wille. QCEC: A JKQ tool for quantum circuit equivalence checking. *Software Impacts*, 7:100051, 2021.

[12] Lukas Burgholzer and Robert Wille. Handling non-unitaries in quantum circuit equivalence checking. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pages 529–534, 2022.

[13] California Institute of Technology. dd: Binary decision diagrams in pure Python and Cython wrappers of CUDD, Sylvan, and BuDDy. `https://github.com/tulip-control/dd`, 2023. Version 0.6.0, BSD-3-Clause.

[14] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. arXiv:quant-ph/0410184, 2004.

[15] Rolf Drechsler, Andisheh Sarabi, M. Theobald, Bernd Becker, and Marek A. Perkowski. Efficient representation and manipulation of switching functions based on ordered Kronecker functional decision diagrams. In *Proceedings of the 31st ACM/IEEE Design Automation Conference (DAC)*, pages 415–419. ACM, 1994.

[16] Rolf Drechsler, M. Theobald, and Bernd Becker. Fast OFDD-based minimization of fixed polarity Reed–Muller expressions. *IEEE Transactions on Computers*, 45(11):1294–1299, 1996.

[17] Craig Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, 2018.

[18] P. Gupta, Abhinav Agrawal, and Niraj K. Jha. An algorithm for synthesis of reversible logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 25(11):2317–2330, 2006.

[19] Xin Hong, Yuan Feng, Sanjiang Li, and Mingsheng Ying. Equivalence checking of dynamic quantum circuits. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2022.

[20] Wei-Jia Huang, Christophe Chareton, Yu-Fang Chen, Kai-Min Chung, Min-Hsiu Hsieh, Alfons Laarman, and Jingyi Mei. Equivalence checking of quantum circuits via path-sum and weighted model counting. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, pages 419–439. Springer, 2026.

[21] Daniela Kaufmann and Jérémy Berthomieu. Extracting linear relations from Gröbner bases for formal verification of and-inverter graphs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS, pages 355–374, 2025.

[22] Daniela Kaufmann and Armin Biere. Improving AMulet2 for verifying multiplier circuits using SAT solving and computer algebra. *International Journal on Software Tools for Technology Transfer*, 25(2):133–144, 2023.

[23] Jingyu Ke, Jingyang Li, and Guoqiang Li. Irene: Equivalence checking of hybrid quantum programs via structure-preserving symbolic reduction. arXiv:2609.36065 [cs.PL], 2026.

[24] U. Kebschull, E. Schubert, and W. Rosenstiel. Multilevel logic synthesis based on functional decision diagrams. In *Proceedings of the European Conference on Design Automation (EDAC)*, pages 43–47. IEEE Computer Society Press, 1992.

[25] Vadym Kliuchnikov, Michael Beverland, and Adam Paetznick. Stabilizer circuit verification. arXiv:2309.08676 [quant-ph], 2023.

[26] Vadym Kliuchnikov, Adam Paetznick, and Marcus P. da Silva. Phased outcome-complete simulation. arXiv:2603.24717 [quant-ph], 2026.

[27] Alireza Mahzoon, Daniel Große, and Rolf Drechsler. RevSCA-2.0: SCA-based formal verification of nontrivial multipliers using reverse engineering and local vanishing removal. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(5):1573–1586, 2022.

[28] Dmitri Maslov, Gerhard W. Dueck, and D. Michael Miller. Techniques for the synthesis of reversible Toffoli networks. *ACM Transactions on Design Automation of Electronic Systems*, 12(4):42, 2007.

[29] Shin-ichi Minato. Zero-suppressed BDDs for set manipulation in combinatorial problems. In *Proceedings of the 30th ACM/IEEE Design Automation Conference (DAC)*, pages 272–277. ACM, 1993.

[30] Shin-ichi Minato. Implicit manipulation of polynomials using zero-suppressed BDDs. In *Proceedings of the European Design and Test Conference (ED&TC)*, pages 449–454, 1995.

[31] Shin-ichi Minato. Zero-suppressed BDDs and their applications. *International Journal on Software Tools for Technology Transfer*, 3(2):156–170, 2001.

[32] Alan Mishchenko, Satrajit Chatterjee, Robert Brayton, and Niklas Eén. Improvements to combinational equivalence checking. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 836–843. IEEE, 2006.

[33] Munich Quantum Toolkit. MQT QCEC: A tool for quantum circuit equivalence checking. `https://github.com/munich-quantum-toolkit/qcec`, 2026. Version 3.11.0, MIT license; test circuits taken from commit 98d6784.

[34] Jérome Ricciardi, Sébastien Bardin, Christophe Chareton, and Benoît Valiron. Quantum circuit equivalence checking: A tractable bridge from unitary to hybrid circuits. arXiv:2511.22523 [quant-ph], 2025.

[35] Fabio Somenzi. *CUDD: CU Decision Diagram Package, Release 3.0.0.* University of Colorado at Boulder, 2015. Mirror: `https://github.com/cuddorg/cudd`, tag cudd-3.0.0.

[36] The BRiAl developers. BRiAl: successor to PolyBoRi. `https://github.com/BRiAl/BRiAl`, 2025. Version 1.2.15, GPL-2.0-or-later.

[37] The passagemath developers. passagemath-brial: Boolean polynomials via BRiAl, modularized SageMath distribution. `https://pypi.org/project/passagemath-brial/`, 2026. Version 10.8.13.

[38] George F. Viamontes, Igor L. Markov, and John P. Hayes. Checking equivalence of quantum circuits and states. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 69–74, 2007.

[39] Ziyuan Wang, Bin Cheng, Longxiang Yuan, and Zhengfeng Ji. FeynmanDD: Quantum circuit analysis with classical decision diagrams. In *Computer Aided Verification (CAV)*, volume 15934 of *LNCS*, pages 28–52, 2025.

[40] Robert Wille, Daniel Große, D. Michael Miller, and Rolf Drechsler. Equivalence checking of reversible circuits. In *39th International Symposium on Multiple-Valued Logic (ISMVL)*, pages 324–330, 2009.

[41] Robert Wille, Daniel Große, Lisa Teuber, Gerhard W. Dueck, and Rolf Drechsler. RevLib: An online resource for reversible functions and reversible circuits. In *38th International Symposium on Multiple Valued Logic (ISMVL)*, pages 220–225, 2008.